

Backtracking

[going back and coming again until condition is satisfied to get the solution (multiple solutions exist))]

Introduction

- Backtracking is an **algorithmic technique** used to solve problems incrementally by **building candidates step by step** and **abandoning (backtracking)** a **candidate** as soon as it is determined that it **cannot lead to a valid solution**.
- It is mainly applied to **combinatorial problems** where we explore all possible solutions but prune unnecessary paths early.

Key Idea of Backtracking

- Construct a solution one step at a time.
- At each step:
 - Check whether the current partial solution is **valid**.
 - If valid, proceed to the next step.
 - If invalid, **undo the choice (backtrack)** and try another option.

Applications of Backtracking

- N-Queen problem
- Subset sum problem
- Graph colouring
- Sudoku solver
- Hamiltonian paths

To get a clear understanding of Backtracking Algorithm and its significance, at first we will recall the concept of Brute Force Method.

Brute Force Method

- The **brute force method** is a **straightforward problem-solving approach** in which **all possible solutions** are generated and checked one by one until the correct or desired solution is found.
- It does **not use any optimization, pruning, or shortcuts**. The logic is simple:

“Try everything and see what works.”

Problem

Q-1: Arrange the following list in ascending order using Brute Force Method.

3	1	2
----------	----------	----------

After that use Backtracking algorithm to compare.



Total 3 places to fill

3 1 2

3 2 1

2 1 3

2 3 1

1 3 2

solution ✓ 1 2 3

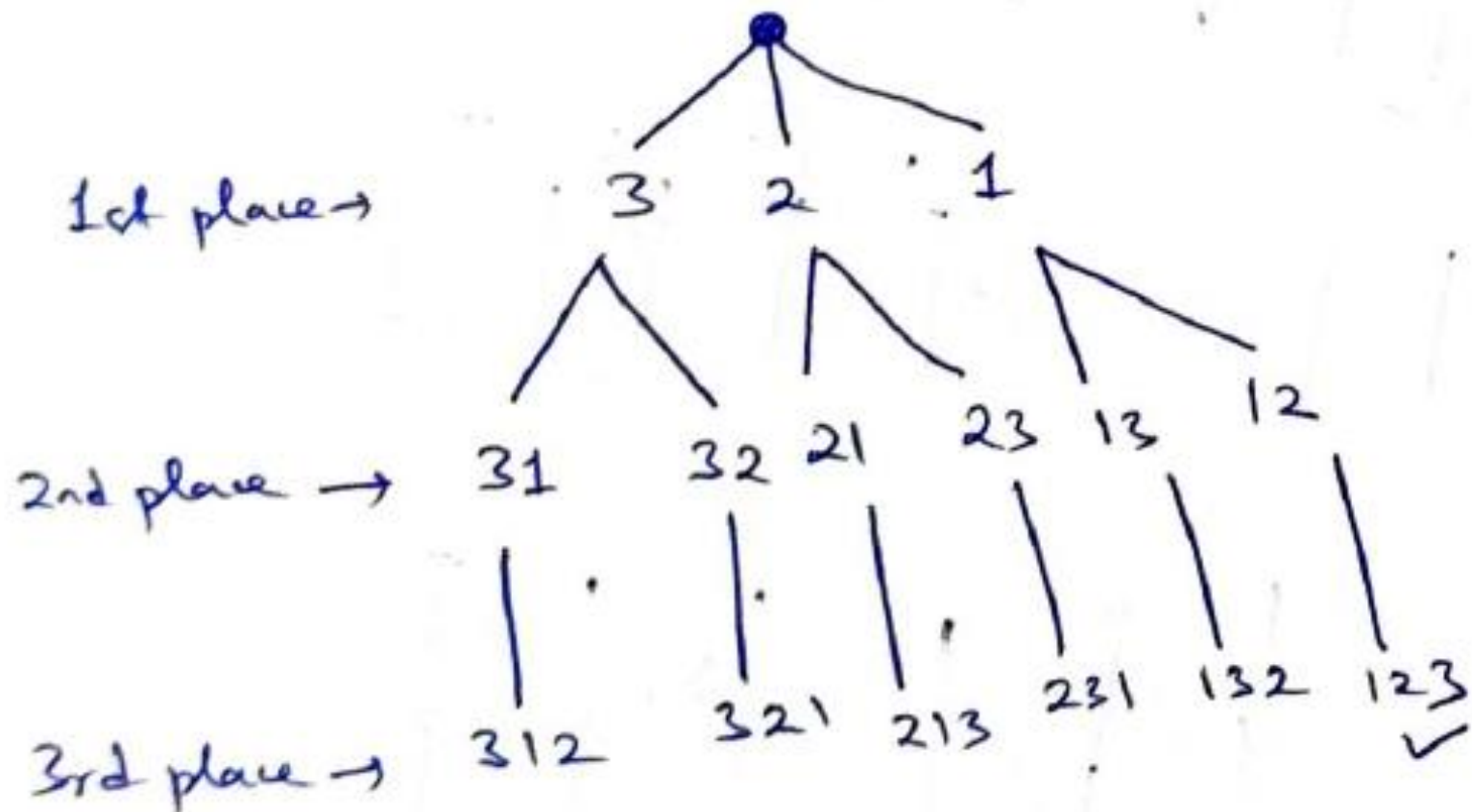
Possible
Arrangements

$$3! = 6$$

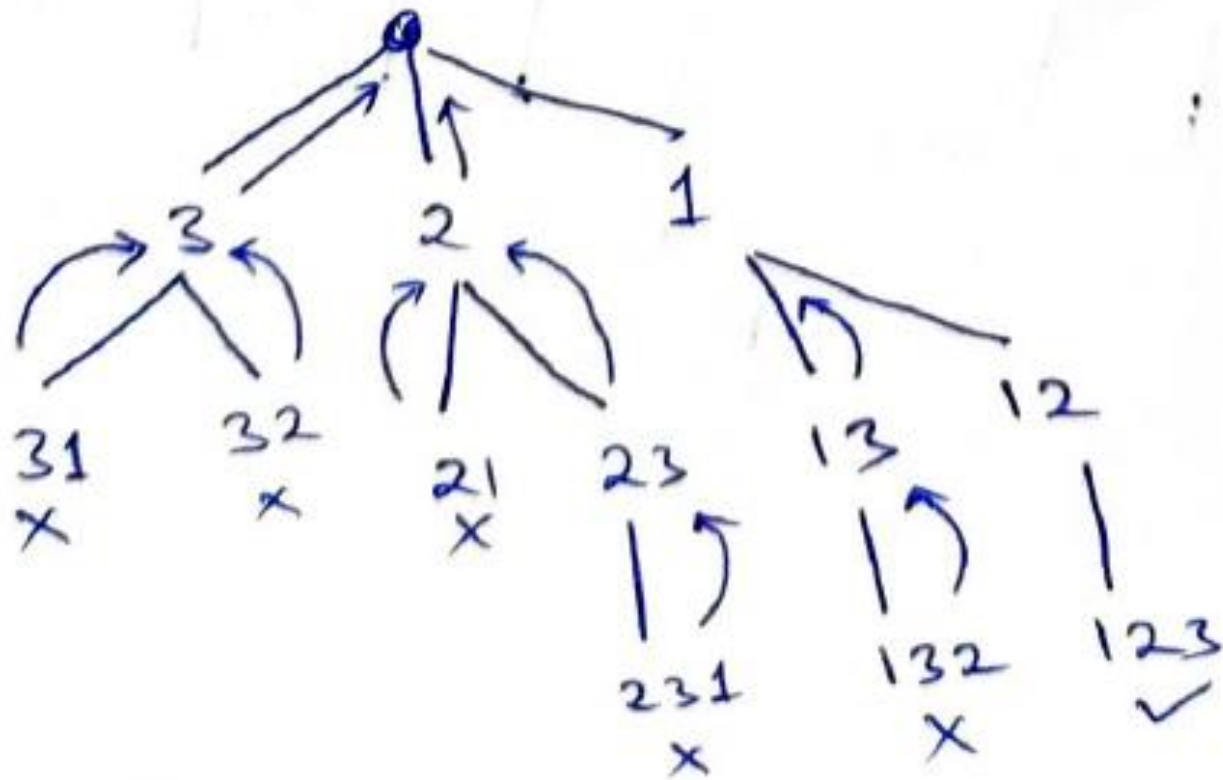
$$\underline{3} \times \underline{2} \times \underline{1}$$

For 'n' elements, we have $n!$ possible arrangements

Tree Structure



Backtracking (Tree Structure)



N-QUEEN PROBLEM

Problem Statement:

The **N-Queen problem** is to place **N queens** on an **$N \times N$ chessboard** such that:

- No two queens attack each other.
- A queen can attack:
 - Same row
 - Same column
 - Same diagonal

Why Backtracking for N-Queen?

- Each row must contain exactly **one queen**.
- We place queens **row by row**.
- If placing a queen in a column leads to a conflict, we **backtrack**.

Constraints for Safe Placement:

A queen at position (row, col) is safe if:

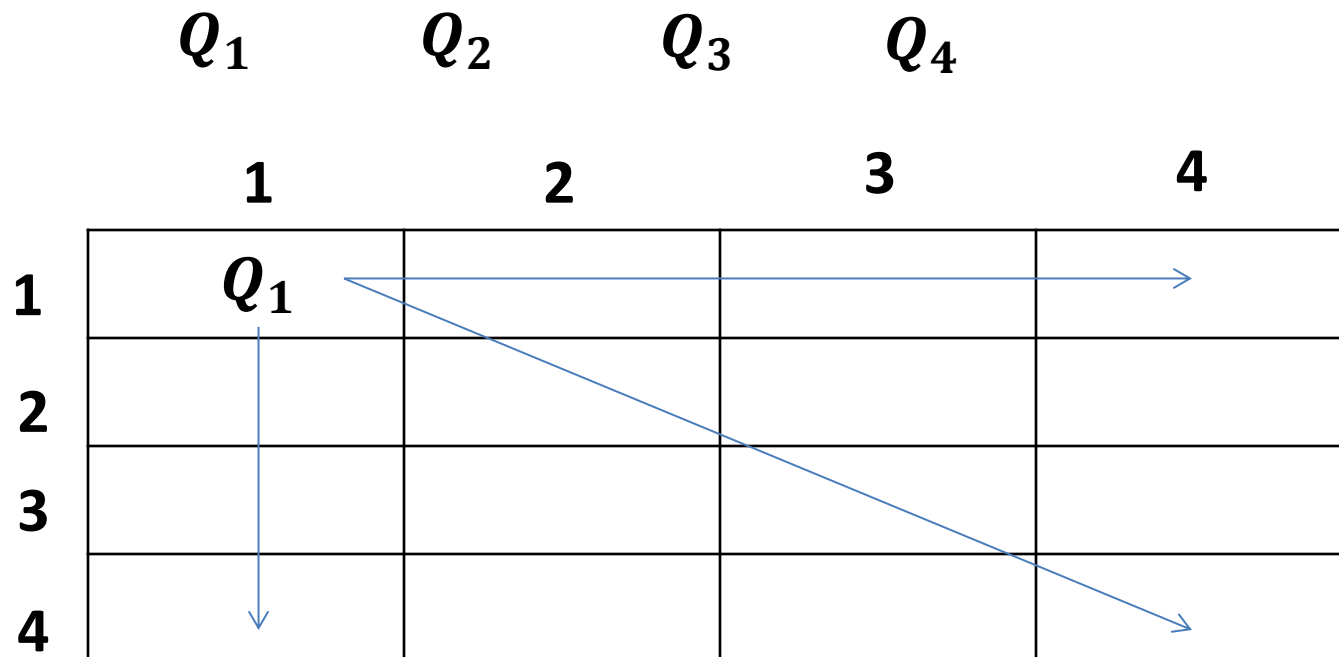
- No queen is placed in the same column.
- No queen is placed in the **upper-left diagonal**.
- No queen is placed in the **upper-right diagonal**.

Algorithm (N-Queen using Backtracking)

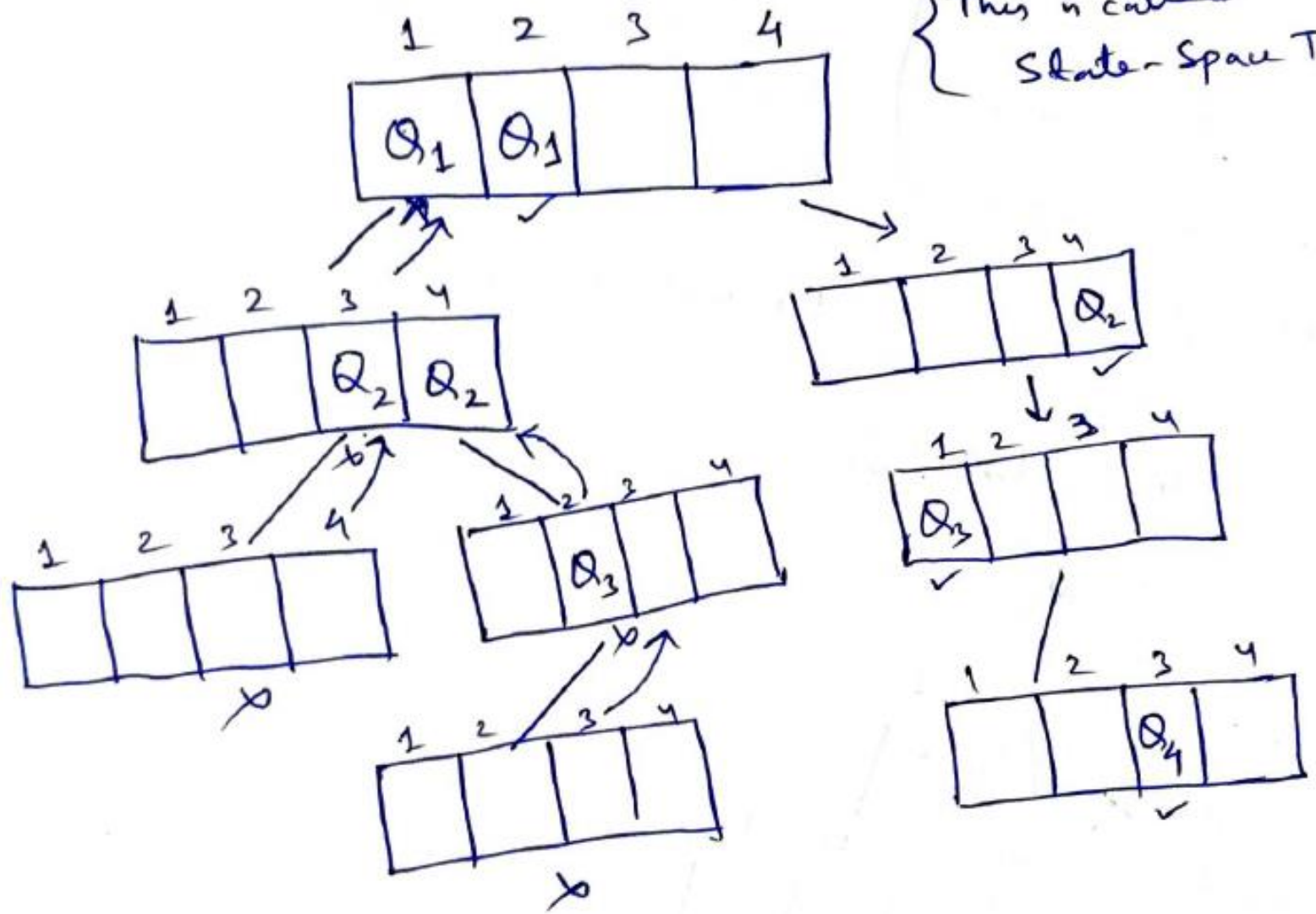
1. Start from row 0.
2. Try placing a queen in each column.
3. Check if the position is safe.
4. If safe:
 - Place the queen.
 - Recur for the next row.
5. If not safe:
 - Try next column.
6. If no column is safe, backtrack.

Working Mechanism

Let's understand N-Queen Problem with the help of backtracking by taking 4-Queens.



{ This is called
State-Space Tree }



	1	2	3	4
1		Q_1		
2				Q_2
3	Q_3			
4			Q_4	

4x4

$$Q_1 = (1, 2)$$

$$Q_2 = (2, 4)$$

$$Q_3 = (3, 1)$$

$$Q_4 = (4, 3)$$

1

		Q_1	
Q_2			
			Q_3
	Q_4		

Mirror

PYTHON IMPLEMENTATION (N-QUEEN)

```
def print_board(board):
    for row in board:
        print(" ".join(str(x) for x in row))
    print()
```

```
def is_safe(board, row, col, n):
    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True
```

```
def solve_n_queen(board, row, n):
    # Base case: All queens placed
    if row == n:
        print_board(board)
        return True

    res = False
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            res = solve_n_queen(board, row + 1, n) or res
            board[row][col] = 0 # Backtrack

    return res

# Driver Code
n = 4
board = [[0 for _ in range(n)] for _ in range(n)]
solve_n_queen(board, 0, n)
```

```
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```


Thank You...